

3600. Maximize Spanning Tree Stability with Upgrades Solved

Hard

Topics

Companies

Hint

You are given an integer n , representing n nodes numbered from 0 to $n - 1$ and a list of `edges`, where `edges[i] = [ui, vi, si, musti]`:

- u_i and v_i indicates an undirected edge between nodes u_i and v_i .
- s_i is the strength of the edge.
- $must_i$ is an integer (0 or 1). If $must_i == 1$, the edge **must** be included in the **spanning tree**. These edges **cannot** be **upgraded**.

You are also given an integer k , the **maximum** number of upgrades you can perform. Each upgrade **doubles** the strength of an edge, and each eligible edge (with $must_i == 0$) can be upgraded **at most** once.

The **stability** of a spanning tree is defined as the **minimum** strength score among all edges included in it.

Return the **maximum** possible stability of any valid spanning tree. If it is impossible to connect all nodes, return -1 .

Note: A **spanning tree** of a graph with n nodes is a subset of the edges that connects all nodes together (i.e. the graph is **connected**) *without* forming any cycles, and uses **exactly** $n - 1$ edges.

Example 1:

Input: $n = 3$, $\text{edges} = [[0, 1, 2, 1], [1, 2, 3, 0]]$, $k = 1$

Output: 2

Explanation:

- Edge $[0, 1]$ with strength = 2 must be included in the spanning tree.
- Edge $[1, 2]$ is optional and can be upgraded from 3 to 6 using one upgrade.
- The resulting spanning tree includes these two edges with strengths 2 and 6.
- The minimum strength in the spanning tree is 2, which is the maximum possible stability.

Example 2:

Input: $n = 3$, $edges = [[0, 1, 4, 0], [1, 2, 3, 0], [0, 2, 1, 0]]$, $k = 2$

Output: 6

Explanation:

- Since all edges are optional and up to $k = 2$ upgrades are allowed.
- Upgrade edges $[0, 1]$ from 4 to 8 and $[1, 2]$ from 3 to 6.
- The resulting spanning tree includes these two edges with strengths 8 and 6.
- The minimum strength in the tree is 6, which is the maximum possible stability.

Example 3:

Input: $n = 3$, $edges = [[0, 1, 1, 1], [1, 2, 1, 1], [2, 0, 1, 1]]$, $k = 0$

Output: -1

Explanation:

- All edges are mandatory and form a cycle, which violates the spanning tree property of acyclicity. Thus, the answer is -1.

Constraints:

- $2 \leq n \leq 10^5$
- $1 \leq \text{edges.length} \leq 10^5$
- $\text{edges}[i] = [u_i, v_i, s_i, \text{must}_i]$
- $0 \leq u_i, v_i < n$
- $u_i \neq v_i$
- $1 \leq s_i \leq 10^5$
- must_i is either 0 or 1.
- $0 \leq k \leq n$
- There are no duplicate edges.

Python:

class Solution:

```
def maxStability(self, N, edges, K):
```

```
    used = 0
```

```
    ans = inf
```

```
    dsu = DSU(N)
```

```
    for u, v, s, m in edges:
```

```
        if m:
```

```
            if not dsu.union(u, v):
```

```
                return -1
```

```
            used += 1
```

```
            ans = min(ans, s)
```

```
    edges.sort(key=lambda e: -e[2])
```

```
    weights = []
```

```
    for u, v, s, m in edges:
```

```
        if m == 0:
```

```
            if dsu.union(u, v):
```

```
                used += 1
```

```
                weights.append(s)
```

```

for i in range(min(K, len(weights))):
    weights[~i] *= 2

```

```

if used != N - 1:
    return -1
return min((ans, *weights))

```

```

class DSU:
    def __init__(self, N):
        self.par = list(range(N))
        self.sz = [1] * N

    def find(self, x):
        if self.par[x] != x:
            self.par[x] = self.find(self.par[x])
        return self.par[x]

    def union(self, x, y):
        xr, yr = self.find(x), self.find(y)
        if xr == yr:
            return False
        if self.sz[xr] < self.sz[yr]:
            xr, yr = yr, xr
        self.par[yr] = xr
        self.sz[xr] += self.sz[yr]
        return True

```

JavaScript:

```

/*****🤪*****/
const maxStability=(n,t,r)=>{let i=new Int32Array(n).fill(-1),e=n,f=1/0;for(let[n,r,o,u]of t){if(u){if(!union(n,r,i))return-1;--e,f=Math.min(f,o)}t.sort(((n,t)=>t[2]-n[2]));for(let n=0;n<t.length&&e>1;++n){let[o,u,l,a]=t[n];if(union(o,u,i)){e-1<=r&&(l*=2),f=Math.min(f,l),--e}}return e>1?-1:f};function union(n,t,r){function i(n){for(;r[n]!==n;)n=r[n];return n}if(r[n]<0&&r[t]<0)return r[n]=n,r[t]=n,!0;if(r[n]>=0&&r[t]>=0){let e=i(r[n]),f=i(r[t]);return r[e]=f,e!==f}return r[t]>=0?r[n]=t:r[t]=n,!0}

```

Java:

```

class Solution {
    public int maxStability(int n, int[][] edges, int k) {
        par = new int [n];
        rank = new int[n];
        for(int i = 0;i<n;i++){
            par[i] = i;
        }
    }
}

```

```
int comp = n;
```

```
Arrays.sort(edges,(a,b)->(b[3]==a[3]?Integer.compare(b[2],a[2]):Integer.compare(b[3],a[3])));
```

```
int minOne = (int)1e9;
```

```
int minZero = (int)1e9;
```

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
for(int [] e: edges){
```

```
    boolean union = union(e[0],e[1]);
```

```
    if(e[3]==1 && !union) return -1;
```

```
    if(union){
```

```
        comp--;
```

```
        if(e[3]==0){
```

```
            pq.add(e[2]);
```

```
        }else{
```

```
            minOne = Math.min(minOne,e[2]);
```

```
        }
```

```
    }
```

```
}
```

```
while(!pq.isEmpty()){
```

```
    if(k-->0){
```

```
        minZero = Math.min(minZero, pq.poll()*2);
```

```
    }else{
```

```
        minZero = Math.min(minZero,pq.poll());
```

```
    }
```

```
}
```

```
if(comp!=1) return -1;
```

```
return Math.min(minOne, minZero);
```

```
}
```

```
int par[];
```

```
int rank[];
```

```
boolean union(int u, int v){
```

```
    int parU = get(u);
```

```
    int parV = get(v);
```

```
    if(parU==parV) return false;
```

```
    if(rank[parU]>=rank[parV]){
```

```
    par[parV] = parU;
    if(rank[parU]==rank[parV]) rank[parU]++;
}else{
    par[parU] = parV;
}
return true;
}
```

```
int get(int u){
    if(par[u]==u) return u;

    return par[u] = get(par[par[u]]);
}
}
```